

# Algoritmos Recursivos

Projeto e corretude

# Motivação

- Algoritmos iterativos avançam para a solução em cada iteração
- Divisão e conquista:
  - Divida a tarefa em partes disjuntas
  - Resolva as partes separadamente
  - Combine as soluções das subtarefas para obter a solução da tarefa original
- Quando as subtarefas são diferentes, teremos subrotinas mais simples
- Quando são similares ao problema original, temos um algoritmo recursivo

# Algoritmo recursivo

```
algoritmo exemplo(n)
  se n = 0 // caso base
    imprima "Oi"
  senão
    x = 2*n // var. local
    exemplo(n-1)
```

Stack frame:

n = 1

x = 2

Stack frame:

n = 2

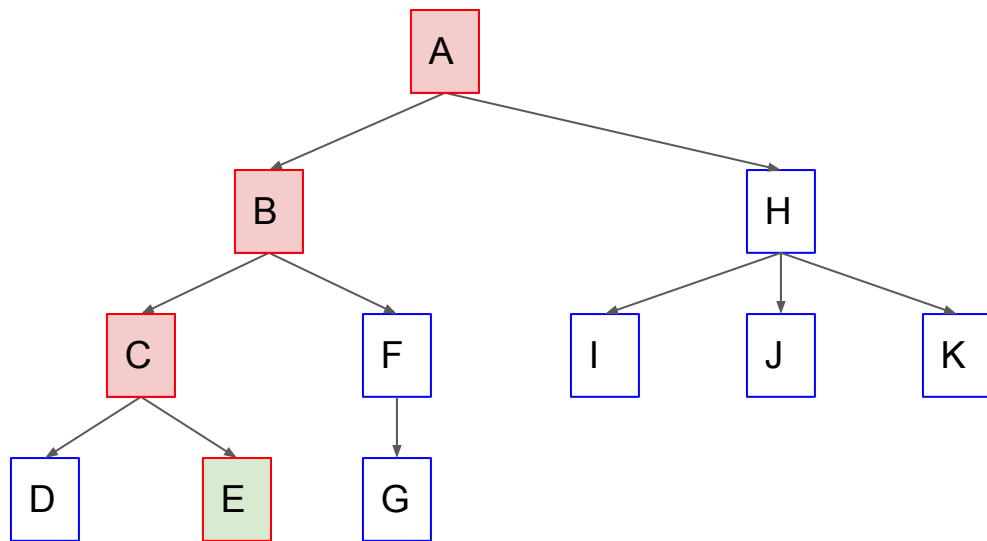
x = 4

Stack frame:

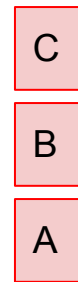
n = 3

x = 6

# Algoritmo recursivo



Árvore de stack frames



Pilha de stack frames

# Algoritmos recursivos - projeto por indução

- Confuso entender o algoritmo pelo código ou pilha/árvore de stack frames
- Projeto por indução é mais simples:
  - Se a instância for suficientemente pequena, resolva (caso base da indução)
  - Senão
    - Construa subinstâncias menores do mesmo problema
    - Suponha que as chamadas recursivas retornam as soluções destas subinstâncias (hipótese de indução)
    - Combine estas subsoluções para construir a solução da instância original
- Não se preocupe em como
  - as chamadas recursivas constróem suas subsoluções
  - a instância recebida foi construída
  - Ou seja, concentre-se apenas em resolver a sua instância usando as subsoluções.

# Algoritmos recursivos - projeto por indução

- Confie no método ao projetar: veremos que corretude decorre de indução
- Parece argumento circular assumir que nosso algoritmo já existe
  - Na verdade é uma sequência de problemas cada vez menores, até o caso base
- Projeto por indução permite projetar pensando em um único passo
- Iterativos vão das instâncias pequenas para as grandes
  - Recursivos vão das instâncias grandes para as pequenas
- Vantagens dos recursivos
  - Às vezes é mais simples projetar de trás para frente (maior para menor)
  - Permite utilizar a solução de várias subinstâncias, formando árvore ao invés de sequência

# Passos do projeto por indução

- Especificação: pré-condições e pós-condições.
  - Ainda mais importante nos recursivos, pois deve garantir as pré-condições nas subinstâncias, e depende do fato das pós-condições serem satisfeitas nas subsoluções
- Tamanho: medida de tamanho da instância
- Entrada geral: considere uma instância grande e geral
  - Assuma que chamadas recursivas retornam subsoluções que atendem pós-condições, desde que você forneça subinstâncias menores (medida tamanho) que satisfaçam as pré-condições
  - Obtenha e combine as subsoluções, para produzir a solução da instância original

# Passos do projeto por indução

- Generalização do problema
  - Pode ser necessário receber mais informação da entrada (pré-condições mais forte)
    - Neste caso você tem que passar esta informação nas chamadas recursivas
  - Pode ser necessário receber mais informação nas subsoluções (pós-condições mais forte)
    - Neste caso você tem que retornar também esta informação adicional
  - Evite passar muita informação: deixe o problema o mais natural possível
  - Pré-condições e pós-condições cumprem o papel do invariante do loop
- Minimização do número de casos
  - Algoritmo deve funcionar para toda entrada que satisfaça as pré-condições
    - Pode ser necessário ter que tratar vários casos
  - Considere os casos do mais geral para o mais particular
    - Verifique se o caso particular já é atendido pelos casos mais gerais
  - Ex.: Caso geral de árvore binária: folha ou nó com dois filhos
    - Caso particular: nó com apenas um dos filhos



# Passos do projeto por indução

- Casos base:
  - Resolva por força bruta quando a instância for suficientemente pequena
- Tempo de execução:
  - Obtenha a relação de recorrência e calcule e aproximação  $\theta$
  - Casos mais complicados: árvore de recursão, somando tempo de cada stack frame

# Relação com os tipos de algoritmos iterativos

- Mais da entrada
  - Equivalente recursivo:
    - Chamada recursiva resolve para o  $n-1$  primeiros
    - Adapta a subsolução para incluir o  $n$ -ésimo elemento
  - Neste caso geralmente é melhor usar o iterativo
    - Stack overflow (falta de memória): tamanho da pilha de stack frames chega até  $n$
  - Pode ser mais fácil projetar recursivo, mas implementar a versão recursiva
    - Não sabemos como avançar para a solução, mas sabemos como acrescentar um elemento em uma solução parcial
  - Recursivo é vantajoso se dividirmos instâncias em várias partes com  $\sim$  o mesmo tamanho
    - Tamanho da pilha de stack frames fica da ordem do  $\log n$  (altura da árvore de recursão)

# Relação com os tipos de algoritmos iterativos

- Mais da saída
  - Equivalente recursivo:
    - Chamada recursiva constrói saída faltando apenas uma parte
    - Acrescenta a parte que está faltando
  - Da mesma forma que o tipo “mais da entrada”:
    - Melhor usar recursivo para construir várias partes com  $\sim$  mesmo tamanho
    - Mas o projeto por indução pode ajudar a projetar o algoritmo iterativo
- Estreitando o espaço de busca
  - Dividimos o espaço de busca em partes com  $\sim$  mesmo tamanho
  - Recursivamente realizamos a busca em algumas destas partes

# Checklist para algoritmo recursivos

- Estrutura: geralmente tem a forma abaixo.

```
algoritmo Alg(I)
```

```
    <pre-cond>: tipos e propriedades das partes de I
```

```
    <pos-cond>: tipos e propriedades das partes da saída retornada
```

```
    se I é suficientemente pequena
```

```
        retorne a solução do caso base I
```

```
     $I_1$  = uma parte de I
```

```
     $S_1$  = Alg( $I_1$ )
```

```
     $I_2$  = outra parte de I
```

```
     $S_2$  = Alg( $I_2$ )
```

```
    S = combine  $S_1$  e  $S_2$ 
```

```
    retorne S
```

# Checklist para algoritmo recursivos

- Especificação: defina claramente as pré-condições e as pós-condições.
- Variáveis
  - Entenda bem a função de cada variável
  - Documente e use bons nomes
  - Pode retornar uma mensagem “não existe solução”, se for o caso
  - Lembre de indicar a variável que recebe o retorno das chamadas recursivas ( $S_1$  e  $S_2$ )
  - Evite ao máximo acrescentar informação na entrada/saída (reforço da pré/pós-condição)
  - Não use variáveis globais: efeitos colaterais difíceis de rastrear
  - Passagem de parâmetro por cópia
    - Chamada recursiva não alteram as variáveis passadas por parâmetro
    - Na implementação pode não ser viável copiar estruturas grandes
      - Neste caso toda mudança deve ser desfeita antes de retornar
  - Geralmente laços e variáveis locais são desnecessários: evite criá-los

# Checklist para algoritmo recursivos

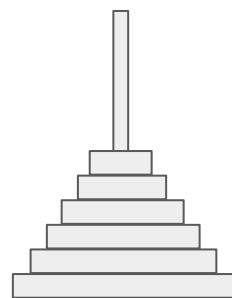
- Tarefas
  - Certifique-se de que o algoritmo funciona para todo tipo de entrada válida
  - Verifique se as subinstâncias:
    - atendem as pré-condições e
    - são menores que a instância original
  - Confie que as subsoluções atendem as pós-condições
    - Não acrescente verificações no algoritmo
    - Não gaste tempo rastreando a árvore de recursão para confirmar
  - Projete a combinação das subsoluções e o caso base

# Corretude do projeto por indução

- $S(n)$ : “O algoritmo recursivo funciona para toda instância de tamanho  $n$ ”
  - “Toda solução retornada para entradas válidas de tamanho  $n$  satisfazem as pós-condições”
  - Queremos provar que  $\forall n \geq 0, S(n)$
  - Assuma que o caso base do algoritmo ocorre quando  $n=0$
- Prova por indução forte em  $n$ 
  - Caso base:  $S(0)$  decorre da corretude do caso base do algoritmo
  - Hipótese de indução: Suponha que  $S(0), S(1), S(2), \dots, S(n-1)$  são verdadeiras.
  - Queremos provar que  $S(n)$  é verdadeira
    - Quando projetamos, assumimos apenas que as subsoluções satisfazem as pós-cond
      - Ou seja,  $S(n)$  depende apenas dos subsoluções satisfazerem as pós-condições
    - Como as subsinstâncias são menores que  $n$ ,
      - por hipótese de indução satisfazem as pós-condições.

# Exemplo: Torres de Hanoi

- Especificação:
  - pré-condições:
    - 3 hastes: origem, destino e auxiliar
    - n discos de tamanhos distintos na haste de origem
  - pós-condições:
    - seq. movimentos dos discos tal que
      - final: os n na haste destino
      - só pode mover disco do topo
      - não pode maior sobre menor
      - pode usar haste auxiliar



Origem



Auxiliar

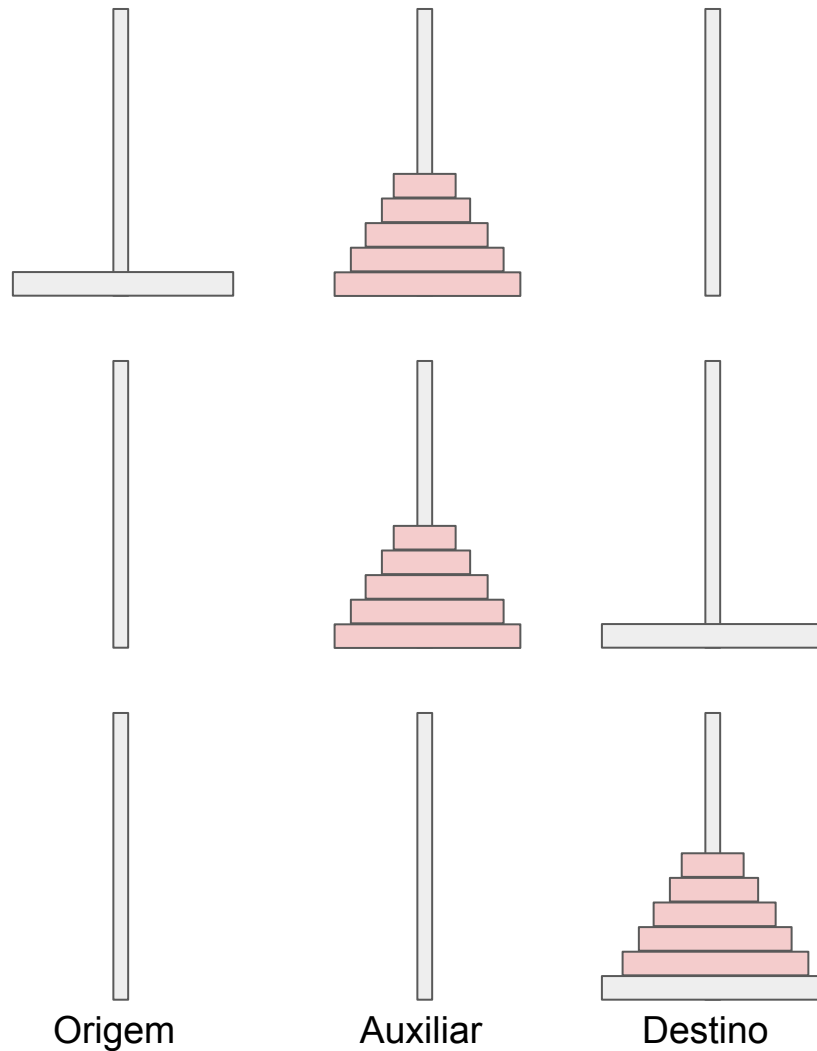


Destino



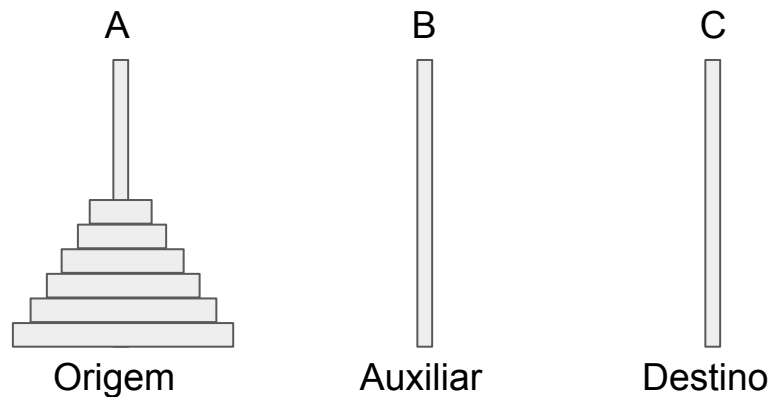
# Exemplo: Torres de Hanoi

- Suponha que recursão fornece movimentos válidos p/ menos de  $n$  discos
- Recursão move  $n-1$  menores p/ auxiliar
- Disco restante da origem para destino
- Recursão move  $n-1$  do auxiliar p/ destino



# Exemplo: Torres de Hanoi

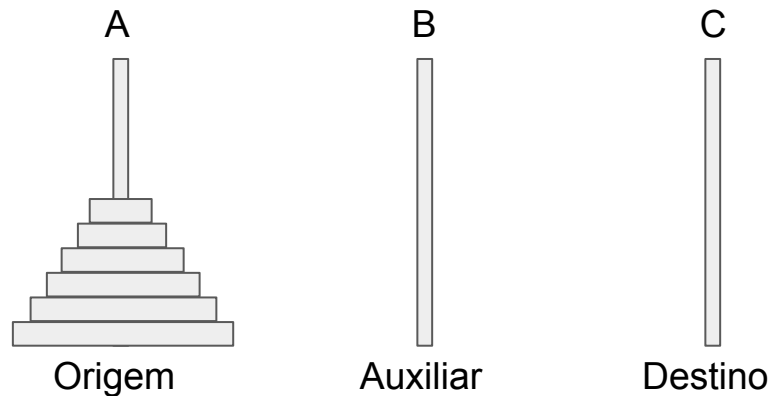
- Reforço das pré-condições:
  - Pode conter discos maiores nas 3 hastes
- Reforço das pós-condições:
  - Apenas os  $n$  menores são movidos
  - Obs.: maiores adicionais são ignorados
- Instância:
  - $n$  discos
  - Origem, Destino, Auxiliar: A, C, B
- Subinstâncias  $I_1$ 
  - $n-1$  discos
  - Origem, Destino, Auxiliar: A, B, C
- Subinstâncias  $I_2$ 
  - $n-1$  discos
  - Origem, Destino, Auxiliar: B, C, A



# Exemplo: Torres de Hanoi

```
algoritmo Hanoi(n, origem, destino, aux)
  se  $n \leq 0$ 
    Nenhum movimento
  senão
    Hanoi(n-1, origem, aux, destino)
    Mova 1 disco da origem p/ destino
    Hanoi(n-1, aux, destino, origem)
```

$$T(n) = 2T(n - 1) + \Theta(1) \in \Theta(2^n)$$



# Exemplo: Mergesort

```
algoritmo Mergesort(A[1..n])  
  se  $n \leq 1$   
    retorne A  
  senão  
    I1 = A[1..|n/2|]  
    S1 = Mergesort(I1)  
    I2 = A[|n/2|+1..n]  
    S2 = Mergesort(I2)  
    retorne Intercala(S1,S2)
```

$$T(n) = 2T(n/2) + \Theta(n) \in \Theta(n \log n)$$

# Exemplo: Subsequência consecutiva máxima (SCM)

- Dada uma sequência  $L[1..n]$  de números
- Determine a maior soma dentre todas as subsequências consecutivas  $L[i..j]$ 
  - Pode ser zero (sequência vazia)
    - Ocorre qndo todos são negativos
- Testar todas as possibilidades:  $\theta(n^2)$
- Ex.: qual seria o maior ganho de uma compra seguida de uma venda de ação?



# Exemplo: Subsequência consecutiva máxima (SCM)

- H.I.: assuma que a recursão fornece soma  $G$  da SCM  $SCM_G$  da sequência  $L[1..n-1]$
- Como incluir  $L[n]$ ?
  - Se  $SCM_G$  é um sufixo
    - Se  $L[n] > 0$ , retorne  $G+L[n]$   
Senão, retorne  $G$
  - Senão, a inclusão de  $L[n]$  pode produzir um sufixo com soma maior que  $G$ 
    - Receber também max soma sufixo (reforço das pós-condições)



# Exemplo: Subsequência consecutiva máxima (SCM)

- H.I.: para  $L[1..n-1]$ , a recursão fornece
  - soma  $G$  da SCM (máximo global)
  - soma  $S$  do sufixo de maior soma (sufixo máximo)
- Se  $S + L[n] > G$ 
  - Retorne  $(S+L[n], S+L[n])$
- Senão, se  $S+L[n] > 0$ 
  - Retorne  $(G, S+L[n])$
- Senão,
  - Retorne  $(G, 0)$
- Caso base:  $n=0$  (seq. vazia)
  - Retorne  $(0, 0)$



# Exemplo: Subsequência consecutiva máxima (SCM)

```
algoritmo SCM(L,n)
  se n <= 0
    retorne (0,0)
  senão
    (G,S) = SCM(L,n-1)
    se S+L[n] > G
      retorne (S+L[n], S+L[n])
    senão, se S+L[n] > 0
      retorne (G, S+L[n])
    senão
      retorne (G, 0)
```

Pilha de stack frames atinge n elementos.  
Melhor transformar em iterativo.





# Exemplo: Subsequência consecutiva máxima (SCM)

```
algoritmo SCM(L,n)
  G = 0
  S = 0
  para i de 1 até n
    se S+L[i] > G
      G = S+L[i]
      S = S+L[i]
    senão, se S+L[i] > 0
      S = S+L[i]
    senão
      S = 0
  retorne G
```

